

MIE 1626 — Summer 2026 — Test Reference Sheet

This sheet is distributed in advance and is the only material allowed at the test (the test is *open handout*, not open book). It lists the R syntax used in the course so that you do not need to memorize it. You should still know *what* each function does and *when* to use it; the sheet only spares you having to remember the spelling.

Functions and control flow

`f <- function(x, y) { ... return(...) }`
 Define a function. The body is in curly braces; the returned value comes from `return()` or, if there is no explicit `return`, the value of the last expression.

`if (cond) { ... } else { ... }`
 Conditional.

Vectors

`c(1, 2, 3)`
 Build a vector.

`v[3]`
 One element (R is 1-indexed).

`v[2:5]`
 A range.

`v[v > 0]`
 Boolean mask: keep the elements where the mask is TRUE.

`&, |, !`
 Element-wise and, or, not.

`sapply(v, f)`
 Apply `f` to every element of `v`; return a vector.

`replicate(n, expr)`
 Run `expr` `n` times; return the results.

`mean(v), sum(v), length(v)`
 Standard reductions.

`which.max(v)`
 Index of the largest element.

Random sampling and simulation

`sample(x, size, replace = FALSE)`
 A random sample of `size` values drawn from `x`, with or without replacement.

`rbinom(n, size, prob)`
`n` draws from a Binomial distribution with `size` trials and success probability `prob`. For a single Bernoulli trial, use `size = 1`.

`rnorm(n, mean = 0, sd = 1)`
`n` draws from a Normal distribution.

`plogis(z)`
 The logistic / sigmoid function: $1 / (1 + \exp(-z))$. Inverse of the logit.

Data frames

`df$col`
 One column as a vector.

`df[row, col]`
 Subset by row and column.

`df[df$x > 0, ]`
 Subset the rows by a boolean mask. Note the trailing comma.

`nrow(df), ncol(df)`
 Row and column counts.

`head(df), tail(df)`
 First or last rows.

The pipe and the tidyverse

`x %>% f()`
 The pipe: equivalent to `f(x)`. Chains compose: `x %>% f() %>% g()` is `g(f(x))`.

`filter(df, cond)`
 Keep rows where `cond` is TRUE.

`select(df, col1, col2)`
 Keep only the named columns.

`arrange(df, col)`
 Sort rows ascending. `arrange(df, desc(col))` sorts descending.

`rename(df, new = old)`
 Rename a column.

`mutate(df, new_col = expr)`
 Add or replace a column.

`group_by(df, col)`
 Group rows for the next `summarize` or `mutate`.

`summarize(df, stat = f(col))`
 Collapse a (possibly grouped) data frame to one row per group.

`distinct(df, col)`
 Unique rows by column.

`n_distinct(col)`
 Count of unique values.

Linear regression

`lm(y ~ x1 + x2, data = df)`
 Fit a linear regression of `y` on `x1` and `x2`.

`summary(model)`

Print the regression summary: coefficient estimates, standard errors, t-values, p-values, residual standard error, R-squared, F-statistic.

```
predict(model, newdata)
```

Predicted y for the rows of `newdata`.

```
summary(model)$r.squared
```

Pull out R-squared directly. Look for **Multiple R-squared** near the bottom of `summary(model)`.

Logistic regression

```
glm(y ~ x1 + x2, family = binomial, data = df)
```

Fit a logistic regression.

```
predict(model, newdata, type = "response")
```

Predicted probabilities for the rows of `newdata`. Without `type = "response"` you get the linear predictor; pass it through `plogis()` to convert it to a probability.

Train / validation / test split

The index-sampling pattern:

```
n      <- nrow(df)
idx    <- sample(1:n, size = floor(0.7 * n))
train  <- df[idx, ]
val    <- df[-idx, ]
```

For a three-way split, sample a second index from the leftover rows.

Diagnostic plots

```
plot(model)
```

Cycles through residuals-vs-fitted, normal Q-Q, scale-location, and residuals-vs-leverage.

Reading a regression table

In `summary(model)`, each row of the **Coefficients** block has columns **Estimate**, **Std. Error**, **t value** (or **z value** for `glm`), and **Pr(>|t|)** (or **Pr(>|z|)**). An approximate 95% confidence interval for a coefficient is **Estimate** $\pm$ 1.96 * **Std. Error**. The line beginning **Multiple R-squared** near the bottom is the R-squared.